

Software Requirements Specification (SRS)

Simple Appointment Record System

1. Project Overview

Build a simple web application for managing appointment records.

The application should have **two pages**:

1. **Appointment Entry Page** — User enters appointment details.
2. **Appointment List Page** — Displays saved records with options to edit, delete, and change the appointment status.

The main purpose of this project is to practice **CRUD operations**.

Use one of the following stacks:

Option A: Python Full Stack

- Backend: Python with FastAPI / Flask / Django REST Framework
- Frontend: React / Vue / equivalent
- Database: PostgreSQL / MySQL

Option B: JavaScript Full Stack

- Backend: Node.js with Express / Fastify / NestJS
- Frontend: React / Vue / equivalent
- Database: PostgreSQL / MySQL

The application must expose REST APIs for backend operations.

2. Appointment Details

Each appointment record contains:

Field	Description
Name	Customer/patient name
Email	Customer/patient email
Phone	Contact number
Appointment Date	Date of appointment
Reason	Reason for the appointment
Status	Pending or Confirmed

3. Page 1 — Add Appointment

The first page should contain a simple form.

Fields

Name

[____Text Field ____]

Email

[____Text Field ____]

Phone

[____Number ____]

Appointment Date

[Select Date]

Reason

[_____ Text Field _____]

[Save Appointment]

On clicking Save

The system should:

1. Validate the required fields.
2. Save the appointment record to the database.
3. Set the initial status as **Pending**.
4. Navigate to the Appointment List page.

4. Page 2 — Appointment List

The second page displays all saved appointment records.

Example:

Appointments Details

Name	Email	Phone	Appointment_Date	Reason	Status
Arun	anbu@mail.com	9876543210	20/09/26	Checkup	Pending
Priya	priay@mail.com	9876543211	21/09/26	Review	Confirmed

Each record should have:

[Edit] [Delete] [Pending / Confirmed]

5. Edit Appointment

When the user clicks **Edit**:

Appointment Details

↓

Edit Form

↓

Modify details

↓

[Update Appointment]

The user should be able to edit:

- Name
- Email
- Phone
- Appointment Date
- Reason

After clicking **Update**, the updated record should appear on the Appointment List page.

6. Delete Appointment

When the user clicks **Delete**:

Delete Appointment?

Are you sure you want to delete this record?

[Cancel] [Delete]

If confirmed, the record should be removed from the database and disappear from the list.

7. Appointment Status

Every appointment should have one of two statuses:

PENDING

CONFIRMED

New appointments should automatically start as:

PENDING

On the second page, the user can click the status button to change:

Pending → Confirmed

and, if required:

Confirmed → Pending

8. CRUD Requirements

The project should demonstrate all four CRUD operations.

Create

Page 1 → Enter details → Save → Database

Read

Database → Page 2 → Display appointments

Update

Page 2 → Edit → Change details → Update

Delete

Page 2 → Delete → Confirm → Remove from database

9. Simple API Requirements

If using a REST API, only these APIs are required:

POST /appointments

GET /appointments

GET /appointments/{id}

PUT /appointments/{id}

DELETE /appointments/{id}

PATCH /appointments/{id}/status

Create

POST /appointments

Get all

GET /appointments

Get one

GET /appointments/{id}

Edit

PUT /appointments/{id}

Delete

DELETE /appointments/{id}

Change status

PATCH /appointments/{id}/status

10. Database Table

Only **one table** is needed.

Table Name : appointments

id

name

email

phone

appointment_date

reason

status

created_at

updated_at

11. Validation

Keep validation basic:

- Name → required
- Email → valid email format
- Phone → required
- Appointment Date → required
- Reason → required
- Status → only PENDING or CONFIRMED